MAD 1 Project Evaluation Report

1. Student Details

- **Name:** Shivani Jain
- **Roll Number:** *24f2005337*
- **Email:** *24f2005337@ds.study.iitm.ac.in*
- **About Me:** I am a student at IIT Madras BS Degree program with a deep interest in web application development and data-driven technologies. I enjoy building meaningful applications that combine learning, analytics, and user experience.

.

2. Project Details

Project Title: TrekPro Portal **(Trekking Management Application)**

Problem Statement: Adventure and trekking organizations require efficient, robust, and centralized web-based architectures to manage outdoor trekking events involving admins, field staff, and participants.

Traditional systems rely heavily on scattered spreadsheets, manual scheduling, or disjointed messaging, which consistently leads to coordination delays, high risks of overbooking, and untrackable booking histories. The objective is to engineer a multi-role authorization web system to safely handle user access, live capacity allocation tracks, search queries, and trek life-cycles.

Approach: The application was structurally engineered using the industry-standard **Model-View-Controller (MVC) Architectural Pattern** natively within Python's Flask framework. To adhere strictly to the academic constraints of the MAD 1 course, the core application features are built entirely via server-side logic and session tracking without depending on client-side JavaScript.

Security interceptions prevent URL manipulation, relational transaction validations block data overbooking, and dynamic HTML interfaces are driven purely using the server-side Jinja2 templating environment.

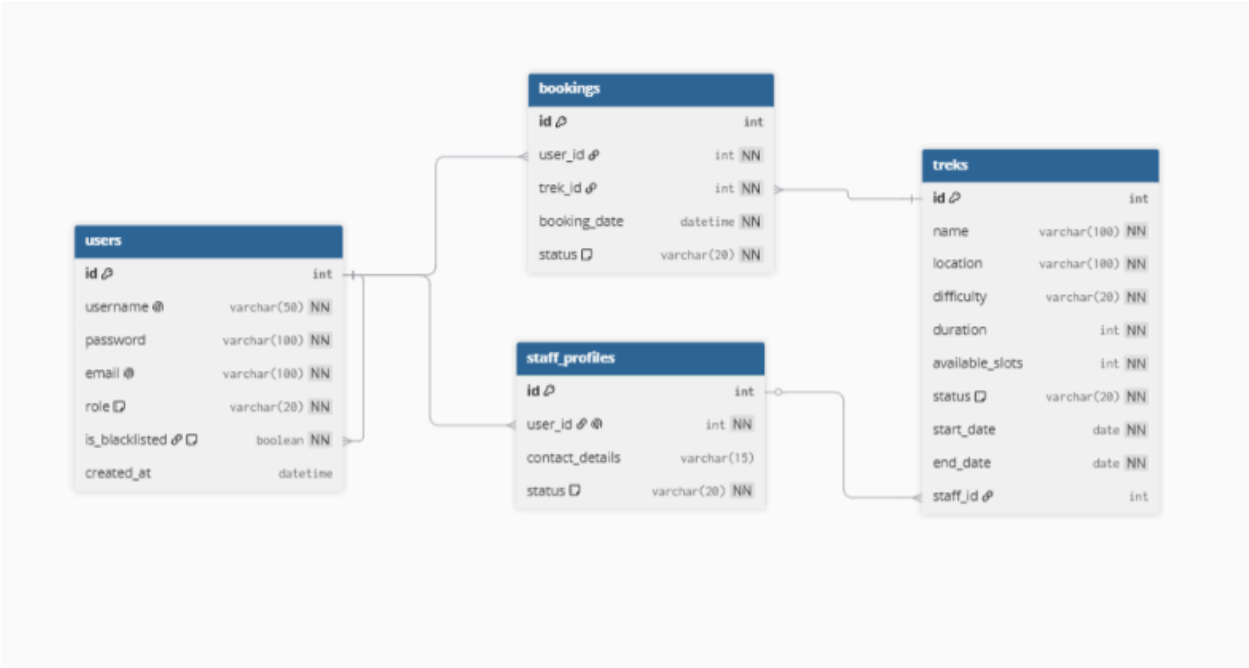
3. AI/LLM Declaration

- I used Gemini to assist in compiling standard markdown documentation files, refining programmatic table relations syntax, and formatting technical parameters into structured layout descriptions.
- The extent of AI/LLM usage is approximately **10–15%**, strictly confined to documentation structural design, syntactic verification, and layout formatting.
- All core architecture components, relational transaction logic rules, controllers routing definitions, and functional UI setups were implemented and debugged manually.

4. Technologies and Frameworks Used

Technology / Library	**Purpose**
Python 3.x	Primary core backend application programming language.
Flask Framework	Micro web framework providing execution controls for routing, server instances, and validation rules.
Flask-SQLAlchemy	Object-Relational Mapper (ORM) system handling the mapping of relational blueprints to database tables.
SQLite 3	Lightweight, high-performance relational storage engine utilizing an auto-generated local database file (trekking.db).
Jinja2	Powerful server-side templating engine handling multi-loop dynamic content rendering, structural layouts inheritance, and conditional UI state distribution.
Bootstrap 5.3	Utility-first frontend component toolkit for crafting a fully responsive, clean layout without manual script interventions.

5. Database Schema / ER Diagram



Relationships Map:

- **One-to-Many:** User → Booking *(A trekker can make multiple trail reservations)*
- **One-to-Many:** Trek → Booking *(A single trek event handles multiple booked ticket lines)*
- **One-to-Many:** StaffProfile → Trek *(An approved guide can be designated to lead multiple scheduled treks)*

6. API Resource Endpoints

The system exposes structured JSON endpoints to facilitate program data access cleanly:

Endpoint	***Method***	***Description***

/api/auth/login	*POST*	*Validates role credentials, maps backend cookies, and boots application state sessions.*
/api/treks/search	*GET*	*Processes query string parameters to filter treks by location, name, or difficulty parameters.*
/api/booking/reserve	*POST*	*Validates storage slot inventory metrics, updates values, and appends a secure reservation row transaction.*
/api/admin/blacklist	*PATCH*	*Alters target account status records dynamically inside the user matrix, breaking active cookies.*

7. Architecture and Implemented Features

Architecture Structure Overview:

The repository follows a clean segregation framework split across application spaces:

- `app.py` *– Application setup orchestrator, extension initialization, and server-side execution launcher.*
- `applications/database.py` *– Initializes the scoped declarative relational database pipelines wrapper objects.*
- `applications/models.py` *– Maps the relational layout tables, foreign keys constraints, and datatype models.*
- `applications/controllers.py` *– Hosts the core routing architecture, access interceptors, business processing logic rules, and CRUD parameter validation layers.*

- `templates/` – *Dynamic server-rendered layouts (`admin_dashboard.html`, `staff_dashboard.html`, `trekker_dashboard.html`, etc.) inheriting from the structural wireframe blueprint `base.html`.*
- `instance/trekking.db` – *Programmatically initialized storage file mapping all active relational data lines.*

Core Implemented Features:

- ***Multi-Role Authentication & Access Authorization:*** *Secure separation parameters mapping distinct UI layers for Admins, Staff, and Users. URL manipulation protection automatically forces a `403 Unauthorized` exit on unverified endpoints.*

- ***Live Capacity Tracking Guardrails:*** *Prevents double-bookings or database over-allocation inside the system loops. If slots drop to zero, backend validations decline requests, and frontend tables auto-toggle to render an immutable "Sold Out" badge.*

- ***Premium UI Controls Without JavaScript (Server-Driven State):***
    1. *Dynamic Content Asset Mapping: Jinja2 string checks parse geographical tags (e.g., Himachal, Uttarakhand) to map matching rich-media layouts using automated URL generators.*
    2. *Auto-Focus Anchor Jumps: Query processing redirections employ HTML target fragments (`#treks-section`) to automatically focus the screen on updated data grids, bypassing tedious page re-scrolling.*

- ***Admin Access Kill-Switch Control:*** *Provides system control keys to review global registries, verify or assign crew personnel, search indices via parameters, and instantly blacklist accounts to protect transaction spaces.*

8. Video Presentation

- ***Drive Link:***
  `https://drive.google.com/file/d/your-actual-drive-link-here`